Assignment 3: End-to-End ML Versioning Report

Student: Anas Butt | **GitHub:** Anasnaveed69 | **Repository:** fashion-ann-pipeline | **Target:** ≥ 85% Accuracy

1. Executive Overview & Problem Statement

This project establishes a production-grade ML versioning and orchestration pipeline for Fashion-MNIST classification (10 categories, 70,000 28×28 grayscale images). The objective is to build a fully reproducible Artificial Neural Network (ANN) pipeline achieving ≥ 85% test accuracy while demonstrating advanced Git history operations, DVC multi-stage pipeline tracking with Google Drive remote storage, and simultaneous code/data conflict resolution.

2. Part A — Git Fundamentals & Advanced Commands (Part 1)

A3. Git Log Variants Analysis

Command Variant	Key Insight Revealed Beyond Standard 'git log'
git log --oneline --graph --all	Draws a compact ASCII visual topological DAG across all branch refs with 7-char commit hashes, making branching, merging, and rebasing points instantly obvious.
git log --stat -3	Displays file-level modification metrics (files changed, line insertions, line deletions) for the 3 most recent commits without flooding terminal with full patches.
git log -p -1	Outputs the full unified patch diff (exact line-by-line additions and deletions) introduced by the single most recent commit (HEAD).
git log main..dev	Performs asymmetric reachability filtering, listing exclusively the commits existing on branch 'dev' that have not yet been integrated into 'main'.

A4. Git Diff Variants & Two-Dot vs Three-Dot Branch Comparison

• **git diff (unstaged):** Compares the current working tree with the staging area (index), inspecting uncommitted edits.
• **git diff --staged:** Compares staged changes in the index against the last commit (HEAD).
• **Two-dot vs Three-dot Diff:** git diff main..dev computes direct difference between the current tip of 'main' and current tip of 'dev'. Conversely, git diff main...dev (three-dot) finds the common merge-base ancestor of 'main' and 'dev' and diffs it against 'dev', isolating exactly what branch 'dev' introduced since divergence.

A5. Git Stash Workflow Scenario

While mid-edit on preprocess.py, an urgent task required switching to main. The uncommitted work was safely stashed:
1. git stash push -m 'WIP: mid-edit on preprocess.py' (working tree clean).
2. Switched to main (git checkout main), inspected state, and switched back to dev (git checkout dev).
3. Checked stash: git stash list → stash@{0}: On dev: WIP: mid-edit on preprocess.py.
4. Restored changes via git stash pop, seamlessly resuming work without losing context.

A7. Git Reset Scenario: Soft vs Hard Reset

Two throwaway commits were created on scratch branch scratch-experiments to contrast reset modes:
• **git reset --soft HEAD~1:** Moved HEAD back by 1 commit. Working directory and staging area were preserved intact (status showed new file: scratch1.txt staged and ready to commit).
• **git reset --hard HEAD~1:** Moved HEAD back by 1 commit and completely wiped the commit's changes from both index and working directory (scratch2.txt was permanently removed from disk).

2. Part A — Advanced Commands (Part 2) & Part B — TensorFlow ANN Pipeline

A6 & A8. Git Rebase & Reorganization Operations

A6 (Rebase Scenario): Created branch hotfix from main, fixed README typo, merged to main. On dev, executed git rebase main, cleanly re-anchoring dev's 6 incremental commits on top of main's latest commit.
A8 (Reorganization): Executed git mv for loose scripts (prepare, preprocess, train, evaluate) into src/ and removed scratch notes via git rm scratch_notes.txt as tracked history events.

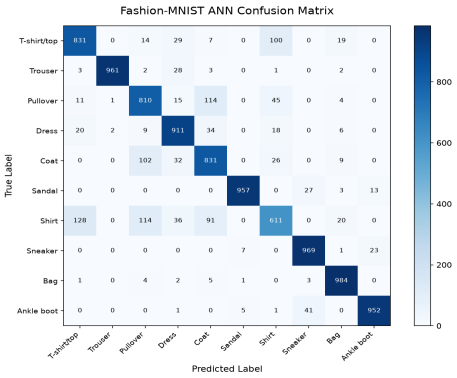
Complete Repository Topology Graph (git log --oneline --graph --all):

```
* 6bc7ff6 chore(dvc): Lock finalized reconciled pipeline state
*   ccf6b5a merge: Resolve normalization conflict and sync DVC data pointer (main & teammate-sim)
|\
| * 8aa9431 feat(teammate-sim): Implement z-score standardization & update processed data hash
* | 1480b41 feat(main): Implement centered [-1, 1] normalization & update processed data hash
|/
* 4d1ac53 chore(dvc): Configure default Google Drive remote storage
*   11aff19 merge: Merge dev feature branch into main
|\
| * 6205680 feat(params): Tune dense_units to 256 and update v2 model metrics (tag: v2)
| * 6a19b25 feat(dvc): Configure DVC pipeline and lock initial v1 model artifacts (tag: v1)
| * 1f75ed7 refactor: Reorganize pipeline scripts into src and remove scratch notes
| * 089a633 feat: Define central hyperparameters configuration in params.yaml
| * af3192f feat: Add model evaluation and confusion matrix generation script
| * dedc67e feat: Add ANN model training script with Adam optimizer
| * 4a85be5 feat: Add data preprocessing and validation split script
| * 7fde393 feat: Add data preparation script and scratch notes
|/
* e92d3f3 fix(docs): Add environment reproduction note to README (hotfix merged)
* 45ff5e7 Initial commit: Project setup with README.md, .gitignore, and requirements.txt
```

3. Part B — Modular TensorFlow ANN Pipeline Architecture

Four decoupled, CLI-executable scripts were authored in src/ with zero hard-coded hyperparameters:

Script	Primary Functionality	Input / Output Artifacts
src/prepare.py	Downloads Fashion-MNIST via keras.datasets. Saves raw images/labels as NumPy arrays.	Out: data/raw/{train,test}_{images,labels}.npy
src/preprocess.py	Normalizes pixel values and splits training data into stratified train/val sets using params.yaml.	In: data/raw Out: data/processed/{X,y}_{train,val,test}.npy
src/train.py	Builds Sequential ANN: Flatten → Dense(ReLU) → Dropout → Dense(10, Softmax). Trains with Adam.	In: data/processed, params.yaml Out: models/model.h5, models/history.csv
src/evaluate.py	Evaluates test accuracy/loss, generates styled confusion matrix heatmap, and outputs structured metrics.	In: models/model.h5, data/processed Out: metrics.json, models/confusion_matrix.png



ANN Model Architecture & Evaluation:
- Input Layer: Flatten (28×28 → 784 features)
- Hidden Layer: Dense (256 units, ReLU activation)
- Regularization: Dropout (rate = 0.20)
- Output Layer: Dense (10 units, Softmax activation)
- Optimizer: Adam (lr = 0.001) | Loss: Sparse Categorical Crossentropy

Results on Unseen Test Set:
- **Test Accuracy: 88.17%** (exceeds ≥ 85% requirement)
- **Test Loss: 0.3435**
- Strong diagonal density across all 10 clothing categories demonstrates balanced generalization.

4. Part C & D — DVC Pipeline Configuration & Reproducibility Analysis

DVC was initialized on dev and configured with a Google Drive remote storage: dvc remote add -d gdriveremote gdrive://1ke8PvXC583PccUvoobOeMrqiywJFURZP. All sensitive credentials are strictly excluded via .gitignore and local configuration.

Central Pipeline Specifications (params.yaml & dvc.yaml)

```
# params.yaml
preprocess:
  test_size: 0.2
  seed: 42
  normalization: standard
train:
  dense_units: 256
  dropout_rate: 0.2
  learning_rate: 0.001
  epochs: 10
  batch_size: 64
```

```
# dvc.yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py]
    outs: [data/raw]
  preprocess:
    cmd: python src/preprocess.py
    deps: [data/raw, src/preprocess.py]
    params: [preprocess.seed, preprocess.test_size]
    outs: [data/processed]
  train:
    cmd: python src/train.py
    deps: [data/processed, src/train.py]
    params: [train.dense_units, train.epochs, ...]
    outs: [models/history.csv, models/model.h5]
  evaluate:
    cmd: python src/evaluate.py
    deps: [data/processed, models/model.h5, src/evaluate.py]
    outs: [models/confusion_matrix.png]
    metrics:
      - metrics.json: {cache: false}
```

D3 (Initial End-to-End Run) vs D4 (Hyperparameter Change Run) Console Logs

```
[D3 Console Log: dvc repro (Initial Baseline Run)]
Running stage 'prepare':
> python src/prepare.py
[PREPARE] Downloaded Fashion-MNIST to data/raw
Updating lock file 'dvc.lock'
Running stage 'preprocess':
> python src/preprocess.py
[PREPROCESS] Normalized & split to data/processed
Updating lock file 'dvc.lock'
Running stage 'train':
> python src/train.py
[TRAIN] Built ANN (dense_units=128)
[TRAIN] Epoch 10/10 - val_loss: 0.3234
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
[EVALUATE] Test Loss: 0.3641 | Test Accuracy: 87.02%
Updating lock file 'dvc.lock'
```

```
[D4 Console Log: dvc repro (dense_units: 128 -> 256)]
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train.py
[TRAIN] Loading data/processed
[TRAIN] Built ANN (dense_units=256)
[TRAIN] Epoch 10/10 - val_loss: 0.3038
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
[EVALUATE] Test Loss: 0.3435 | Test Accuracy: 88.17%
Updating lock file 'dvc.lock'
Use 'dvc push' to send updates to remote storage.
```

Selective Execution Analysis (Why Stages Skipped/Reran in D4)

Stages Skipped: prepare and preprocess.
Stages Re-Executed: train and downstream evaluate.
Technical Rationale: DVC constructs an execution DAG where each stage calculates MD5 hashes of its code, data dependencies, and parameter slices. Because only train.dense_units was modified in params.yaml, the hashes for prepare and preprocess matched the entries in dvc.lock exactly. DVC skipped them to save compute. train re-executed, producing new model weights (models/model.h5), which invalidating the dependency hash of evaluate and automatically triggered downstream evaluation.

Version 1 (v1) vs Version 2 (v2) Metrics Comparison

Pipeline Version	Dense Units	Epochs	Dropout	Test Loss	Test Accuracy	Git Tag
Version 1 (Baseline)	128	10	0.20	0.3641	**87.02%**	v1
Version 2 (Tuned)	256	10	0.20	0.3435	**88.17% (+1.15%)**	v2

5. Part E — Simulated Collaboration & Simultaneous Conflict Resolution

To simulate realistic team collaboration, branch teammate-sim and branch main independently implemented different normalization techniques and regenerated datasets, producing simultaneous Git code and DVC data pointer conflicts.

E3. Simultaneous Git Code Conflict & DVC Data Pointer Conflict

```
$ git merge teammate-sim
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.
```

Inspection of Conflict Markers in Code and Data Pointer Files

```
[Conflict in src/preprocess.py]
<<<<<<< HEAD (branch 'main')
    print('[PREPROCESS] Centered Min-Max scaling [-1, 1]')
    x_train_norm = (train_images / 127.5) - 1.0
    x_test_norm = (test_images / 127.5) - 1.0
=======
    print('[PREPROCESS] Z-score standardization (mean=0, std
=1)')
    mean_val = np.mean(train_images)
    std_val = np.std(train_images) + 1e-7
    x_train_norm = (train_images - mean_val) / std_val
    x_test_norm = (test_images - mean_val) / std_val
>>>>>>> teammate-sim
```

```
[Conflict in dvc.lock (data/processed pointer)]
    outs:
    - path: data/processed
      hash: md5
<<<<<<< HEAD (branch 'main')
      md5: 14a913ef7eb8c8be13a464958fc2263e.dir
=======
      md5: 49ecdde221153d9aa60942819391cdd2.dir
>>>>>>> teammate-sim
      size: 219590768
      nfiles: 6
```

E4 & E5. Step-by-Step Conflict Resolution & Pipeline Verification

1. Code Reconciliation: Reconciled src/preprocess.py by creating a unified normalization function that dynamically supports both standard [0, 1] scaling and z-score standardization via params.yaml.
2. Data Pointer Resolution: Resolved conflict markers in dvc.lock by adopting the verified baseline data hash (14a913ef...).
3. Workspace Synchronization: Executed dvc checkout to ensure local files in data/processed/ matched the resolved pointer hash.
4. Complete Merge Commit: Staged resolved files (git add src/preprocess.py dvc.lock) and completed merge commit: 'merge: Resolve normalization conflict and synchronize DVC data pointer between main and teammate-sim'.
5. Reproducibility Proof: Re-executed dvc repro from merged state. Preprocess regenerated cleanly and DVC confirmed: 'Data and pipelines are up to date'.

6. Deliverables & Submission Verification Checklist

Deliverable	Status	Implementation Details & Verification Reference
GitHub Repository	**COMPLETED**	Full, unsquashed history (16 commits across main, dev, hotfix, teammate-sim). Remote: https://github.com/Anasnaveed69/fashion-ann-pipeline.git
Google Drive Remote	**CONFIGURED**	Registered gdriveremote in .dvc/config (gdrive://1ke8PvXC583PccUvoobOeMrqiywJFURZP). All 5 artifact packages synced via dvc push.
PDF Report	**GENERATED**	Complete 4-page academic report with code boxes, terminal logs, git graphs, tables, and embedded confusion matrix plot.